MERN STACK

E-Commerce Web App

Roman Urdu Mein — Zero Se Hero Tak

CHAPTER 8 | REDUX TOOLKIT | STATE MANAGEMENT | ROMAN URDU

CHAPTER 8

Redux Toolkit — Professional State Management

Store, Slices, useSelector, useDispatch, createAsyncThunk — sab sikhenge!

Is Chapter Mein Aap Seekhenge:

- Redux kya hai aur kyun use karte hain
- Redux Toolkit install aur Store setup
- Slices — actions aur reducers ek saath
- useSelector — state read karna
- useDispatch — actions dispatch karna
- createAsyncThunk — API calls with Redux
- cartSlice — add, remove, update, clear
- authSlice — login, logout, user state

- productSlice — products, loading, error
- Context se Redux mein migration

Chapter 7 Recap

React Router, useParams, useNavigate, Context API, useReducer, Custom Hooks, Protected Routes — yeh sab aana chahiye. Chapter 8 mein Context ko Redux se replace karenge!

TABLE OF CONTENTS

7.0 createAsyncThunk — API Calls

- 7.1 Kya hai aur kyun?
- 7.2 Thunk syntax
- 7.3 pending / fulfilled / rejected
- 7.4 extraReducers
- 7.5 productSlice complete

8.0 productSlice — Products API

- 8.1 fetchProducts thunk
- 8.2 fetchProductById thunk
- 8.3 Slice with extraReducers
- 8.4 Error handling

9.0 Components Update

- 9.1 HomeScreen — Redux se products
- 9.2 CartScreen — Redux cart
- 9.3 Navbar — Redux auth
- 9.4 LoginScreen — dispatch karna

10.0 Summary & Practice

- 10.1 Chapter recap
- 10.2 Practice exercises
- 10.3 Chapter 9 preview

SECTION 1 — Redux Kya Hai?

1.0 — Problem — Context Kaafi Nahi?

Chapter 7 mein humne Context API se kaam chalaya — lekin jaise app badi hoti hai, Context akela kaafi nahi hota. Socho ShopEase mein: products, cart, auth, orders, filters, search — sab ka state manage karna hai. Alag alag Contexts banaao, sab wrap karo — bohat messy ho jaata hai!

Masla	Context mein	Redux mein
State updates debug	Kahan se aaya? Pata nahi	DevTools mein har action ka log
Complex state logic	UseReducer + Context = verbose	createSlice — clean aur concise
Performance	Har context change = re-render	Sirf subscribed component update
Middleware (side effects)	Manually handle	Built-in middleware support
Time-travel debugging	Mumkin nahi	Redux DevTools se possible!
Large team	Conventions ka koi standard nahi	Strict pattern — team-friendly

1.1 — Redux Ka Core Concept

Redux ek **single global store** hai jisme poori app ka state hota hai. Koi bhi component directly state nahi badalta — ek **action dispatch** karta hai, **reducer** action ko dekh ke state update karta hai, aur **subscribed components** automatically update ho jaate hain:

1	**Component**	dispatch(action) call karta hai — 'kya karna hai' batata hai
2	**Store**	Action receive karta hai — reducer ko bhejta hai
3	**Reducer**	Current state + action dekhta hai — naya state return karta hai
4	**Store**	Naya state save karta hai
5	**Components**	useSelector se state read karte hain — automatically re-render

■ TIP

Redux Toolkit (RTK) Redux ka official, opinionated toolset hai. Purana Redux bahut verbose tha — RTK ne sab simplify kar diya. 2023+ mein hamesha RTK use karo — purana Redux nahi!

SECTION 2 — Redux Toolkit Setup

2.0 — Install Karna

```
# Redux Toolkit + React-Redux install karo
npm install @reduxjs/toolkit react-redux

# @reduxjs/toolkit — Redux Toolkit (RTK)
# react-redux     — React ke saath Redux connect karne ke liye
```

2.1 — Store Banana (src/store/store.js)

Store ek central jagah hai jahan saara app state rehta hai. RTK mein **configureStore** se banate hain:

■ src/store/store.js

```
import { configureStore } from '@reduxjs/toolkit';
import cartReducer    from './cartSlice';
import authReducer    from './authSlice';
import productReducer from './productSlice';

const store = configureStore({
  reducer: {
    // Har key ek 'slice' ki state hai
    cart:    cartReducer,
    auth:    authReducer,
    products:productReducer,
  },
  // DevTools development mein auto enable hota hai
  // Middleware mein thunk automatically add hota hai
});

export default store;

// TypeScript users ke liye (optional):
// export type RootState = ReturnType;
// export type AppDispatch = typeof store.dispatch;
```

2.2 — Provider Wrap (src/index.js)

Poori app ko Redux **Provider** mein wrap karo — taki har component store access kar sake:

```
// src/index.js
import { Provider } from 'react-redux';
import store from './store/store';
// ... baaki imports

root.render(
  <React.StrictMode>
    <Provider store={store}>    {/* Redux store provide karo */}
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </Provider>
  </React.StrictMode>
);

// Ab koi bhi component useSelector / useDispatch use kar sakta hai
// Context API Providers ki zaroorat nahi — Redux handle karega!
```

2.3 — Redux DevTools — Debugging Ka Best Friend

Redux DevTools Chrome extension install karo — yeh ek magical debugging tool hai:

Feature	Kya Karta Hai?
Action Log	Har dispatch hua action time ke saath dikh jaata hai
State Diff	Har action ke baad state mein kya badla — clearly dikhai deta hai
Time Travel	Kisi bhi purani state pe wapas jao — bugs dhundna bohat asaan
Action Replay	Actions dobara replay karo — testing ke liye
State Inspector	Current poora state tree dekho — nested objects bhi

■ TIP

Chrome Web Store mein 'Redux DevTools' search karo aur install karo. Phir browser mein F12 -> Redux tab. Yeh tool Redux seekhne aur debug karne mein bahut help karta hai!

SECTION 3 — Slices — Actions + Reducers

3.0 — Slice Kya Hai?

Slice ek feature ka poora state management package hai — initial state, reducers, aur actions sab ek jagah. RTK se pehle yeh alag alag files mein likhna padta tha. Ab **createSlice** se ek call mein sab ho jaata hai:

```
import { createSlice } from '@reduxjs/toolkit';

// Ek simple counter slice — concept samjhne ke liye
const counterSlice = createSlice({
  name: 'counter',          // Slice ka naam — action types mein use hoga
  initialState: { value: 0 }, // Starting state

  reducers: {
    // Har function ek action + reducer dono hai!
    increment: (state) => {
      state.value += 1;  // Direct mutation — Immer handle karta hai
    },
    decrement: (state) => {
      state.value -= 1;
    },
    incrementByAmount: (state, action) => {
      state.value += action.payload;  // payload = dispatch ke saath bhej
a data
    },
    reset: (state) => {
      state.value = 0;
    },
  },
});

// Actions automatically generate hote hain!
export const { increment, decrement, incrementByAmount, reset } = counter
Slice.actions;
```

```
// Reducer export karo — store mein use hoga
export default counterSlice.reducer;
```

3.1 — Immer — Direct Mutation Allowed!

Normal Redux mein hum state mutate nahi kar sakte the — nayi copy banana padti thi. RTK andar se **Immer** library use karta hai jo directly mutate karne deta hai — Immer safely copy banata hai:

```
// ■ Purana Redux — bohat verbose
case 'ADD_ITEM':
  return {
    ...state,
    items: [...state.items, action.payload],  // Manual spread
  };

// ■ RTK with Immer — clean!
addItem: (state, action) => {
  state.items.push(action.payload);  // Direct push — Immer handle karta hai
},

// RTK ke bahar Immer use mat karo — wahan hamesha new state return karo
```

`createSlice({ name, initialState, reducers })`	Ek slice banata hai. name = action type prefix. initialState = starting state. reducers = state update functions.
`action.payload`	dispatch(action(data)) mein jo data bheja — woh payload mein milta hai. Jaise addItem(product) mein product yahan milega.
`slice.actions`	createSlice automatically action creators banata hai — inhe export karo aur dispatch() mein use karo.
`slice.reducer`	Ek single reducer function — store ke reducer object mein add karo.

SECTION 4 — useSelector aur useDispatch

4.0 — Component Redux Se Kaise Milta Hai?

Redux store aur React components ke beech 2 hooks kaam karte hain. Yeh dono **react-redux** se aate hain:

Hook	Kaam	Analogy
useSelector	Store se state READ karo	Dukaan se cheez dekho — kharidna nahi
useDispatch	Action DISPATCH karo — state update	Dukaan ko order do

4.1 — useSelector — State Read Karna

```
import { useSelector } from 'react-redux';

function CartIcon() {
  // Store se cart state lo
  // state.cart — store.js mein 'cart' key pe rakha hai
  const cartItems  = useSelector(state => state.cart.items);
  const totalItems = useSelector(state => state.cart.totalItems);

  // Jab bhi cart state change hoga — yeh component re-render hoga
  return (
    <div>
      Cart ({totalItems} items)
    </div>
  );
}

// Multiple values ek baar mein:
const { items, totalItems, totalPrice } = useSelector(state => state.cart
);
```

4.2 — useDispatch — Action Dispatch Karna

```
import { useDispatch } from 'react-redux';
import { addItem, removeItem } from '../store/cartSlice';
```

```
function ProductCard({ product }) {
  const dispatch = useDispatch();

  const addToCartHandler = () => {
    // Action dispatch karo — cartSlice ka addItem
    dispatch(addItem(product));
    // dispatch(addItem({ ...product, quantity: 1 }));
  };

  const removeHandler = (id) => {
    dispatch(removeItem(id));
  };

  return (
    <div>
      <h3>{product.name}</h3>
      <button onClick={addToCartHandler}>Add to Cart</button>
    </div>
  );
}
```

■ TIP

useSelector ka argument ek selector function hota hai — poora state milta hai, tum decide karo kya chahiye. Sirf jo chahiye woh lo — performance better rahegi. Poora state mat lo agar sirf ek field chahiye!

SECTION 5 — cartSlice — Shopping Cart

5.0 — Cart State Ka Plan

Cart mein yeh sab operations hone chahiyen — add, remove, quantity update, aur clear:

Action	Payload	Kya Hoga?
addItem	product object	Cart mein add — agar pehle se hai quantity badhao
removeItem	product _id	Cart se remove — filter se hatao
updateQuantity	{ id, quantity }	Quantity update — 1 se kam nahi ho sakti
clearCart	koi nahi	Poora cart khaali — array = []

■ src/store/cartSlice.js

```
import { createSlice } from '@reduxjs/toolkit';

// localStorage se cart load karo (page refresh ke baad bhi rahega)
const cartFromStorage = localStorage.getItem('cartItems')
  ? JSON.parse(localStorage.getItem('cartItems'))
  : [];

const initialState = {
  items:      cartFromStorage,
  totalItems: 0,
  totalPrice: 0,
};

// Helper — totals calculate karna
const calcTotals = (state) => {
  state.totalItems = state.items.reduce((sum, i) => sum + i.quantity, 0);
  state.totalPrice = state.items.reduce((sum, i) => sum + i.price * i.qua
ntity, 0);
};

const cartSlice = createSlice({
  name: 'cart',
```

```
  initialState,
  reducers: {

    // ■■ Add Item ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    addItem: (state, action) => {
      const product = action.payload;
      const exists  = state.items.find(i => i._id === product._id);

      if (exists) {
        // Pehle se hai — quantity +1
        exists.quantity += 1;
      } else {
        // Naya — add karo with quantity 1
        state.items.push({ ...product, quantity: 1 });
      }

      calcTotals(state);
      // localStorage mein save karo
      localStorage.setItem('cartItems', JSON.stringify(state.items));
    },

    // ■■ Remove Item ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    removeItem: (state, action) => {
      state.items = state.items.filter(i => i._id !== action.payload);
      calcTotals(state);
      localStorage.setItem('cartItems', JSON.stringify(state.items));
    },

    // ■■ Update Quantity ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    updateQuantity: (state, action) => {
      const { id, quantity } = action.payload;
      const item = state.items.find(i => i._id === id);
      if (item) {
        item.quantity = Math.max(1, quantity);  // 1 se kam nahi
      }
      calcTotals(state);
      localStorage.setItem('cartItems', JSON.stringify(state.items));
```

```
    },

    // ■■ Clear Cart ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
    clearCart: (state) => {
      state.items = [];
      state.totalItems = 0;
      state.totalPrice = 0;
      localStorage.removeItem('cartItems');
    },
  },
});

export const { addItem, removeItem, updateQuantity, clearCart } = cartSlice.actions;

// Selectors — state se specific data nikalna
export const selectCartItems   = state => state.cart.items;
export const selectTotalItems  = state => state.cart.totalItems;
export const selectTotalPrice  = state => state.cart.totalPrice;

export default cartSlice.reducer;
```

SECTION 6 — authSlice — Authentication State

6.0 — Auth State Redux Mein

Chapter 7 mein AuthContext banaya tha — ab usi logic ko Redux slice mein le aate hain. Fayda: DevTools mein login/logout track hoga, aur baaki slices bhi auth state read kar sakenge:

src/store/authSlice.js

```
import { createSlice } from '@reduxjs/toolkit';

// localStorage se user load karo
const userFromStorage = localStorage.getItem('user')
  ? JSON.parse(localStorage.getItem('user'))
  : null;

const tokenFromStorage = localStorage.getItem('token') || null;

const initialState = {
  user:        userFromStorage,
  token:       tokenFromStorage,
  isLoggedIn:  !!userFromStorage,
  isAdmin:     userFromStorage?.role === 'admin',
};

const authSlice = createSlice({
  name: 'auth',
  initialState,

  reducers: {
    // Login / Register ke baad call karo
    setCredentials: (state, action) => {
      const { user, token } = action.payload;
      state.user       = user;
      state.token      = token;
      state.isLoggedIn = true;
```

```
      state.isAdmin    = user?.role === 'admin';

      // localStorage mein save karo
      localStorage.setItem('user',  JSON.stringify(user));
      localStorage.setItem('token', token);
    },

    // Logout
    logout: (state) => {
      state.user       = null;
      state.token      = null;
      state.isLoggedIn = false;
      state.isAdmin    = false;

      localStorage.removeItem('user');
      localStorage.removeItem('token');
    },
  },
});

export const { setCredentials, logout } = authSlice.actions;

// Selectors
export const selectUser      = state => state.auth.user;
export const selectToken     = state => state.auth.token;
export const selectIsLoggedIn= state => state.auth.isLoggedIn;
export const selectIsAdmin   = state => state.auth.isAdmin;

export default authSlice.reducer;
```

■ TIP

Selectors export karna good practice hai — jaise selectUser, selectIsAdmin. Agar state structure badle toh sirf slice file mein update karo — har component mein state => state.auth.user nahi badalna padega!

SECTION 7 — createAsyncThunk — API Calls

7.0 — Async Operations Redux Mein Kyun Mushkil Hain?

Redux reducers **pure synchronous functions** hain — andar async await nahi likh sakte. API call kaise karein? **createAsyncThunk** yeh masla solve karta hai — ek async function banata hai jo automatically 3 actions dispatch karta hai:

State	Kab?	Use Case
pending	API call shuru hui	loading = true dikhao
fulfilled	API call successful — data aaya	loading = false, data save karo
rejected	API call fail — error aaya	loading = false, error dikhao

7.1 — createAsyncThunk Syntax

```
import { createAsyncThunk } from '@reduxjs/toolkit';
import { getProducts } from '../services/api';

// createAsyncThunk(actionName, asyncFunction)
export const fetchProducts = createAsyncThunk(
  'products/fetchAll',    // Action type name — unique hona chahiye
  async (params, thunkAPI) => {
    try {
      const { data } = await getProducts(params?.keyword, params?.page);
      return data;  // Yeh fulfilled action ka payload banega
    } catch (error) {
      // rejectWithValue se custom error bhejo
      return thunkAPI.rejectWithValue(
        error.response?.data?.message || 'Products load nahi hue!'
      );
    }
  }
);
```

```
// Component mein use karna:
// dispatch(fetchProducts({ keyword: 'shoes', page: 1 }))
// params mein jo bheja woh milega
```

7.2 — extraReducers — Thunk States Handle Karna

Thunk ke 3 states ko **extraReducers** mein handle karo — yeh slice ke andar hota hai:

```
const productSlice = createSlice({
  name: 'products',
  initialState: { items: [], loading: false, error: '', total: 0, pages: 1 },
  reducers: {},  // Synchronous actions yahan

  // Async thunk states yahan handle karo
  extraReducers: (builder) => {

    builder
      // ■■ fetchProducts ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
      .addCase(fetchProducts.pending, (state) => {
        state.loading = true;
        state.error   = '';
      })
      .addCase(fetchProducts.fulfilled, (state, action) => {
        state.loading = false;
        state.items   = action.payload.data;
        state.total   = action.payload.total;
        state.pages   = action.payload.totalPages;
      })
      .addCase(fetchProducts.rejected, (state, action) => {
        state.loading = false;
        state.error   = action.payload; // rejectWithValue se aaya
      });
  },
});
```

SECTION 8 — productSlice — Complete

■ src/store/productSlice.js

```
import { createSlice, createAsyncThunk } from '@reduxjs/toolkit';
import { getProducts, getProductById, createProduct,
         updateProduct, deleteProduct } from '../services/api';

// ■■ Async Thunks ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
export const fetchProducts = createAsyncThunk(
  'products/fetchAll',
  async ({ keyword='', page=1, category='' } = {}, { rejectWithValue }) =
> {
    try {
      const { data } = await getProducts(keyword, page, category);
      return data;
    } catch (err) {
      return rejectWithValue(err.response?.data?.message || 'Error!');
    }
  }
);

export const fetchProductById = createAsyncThunk(
  'products/fetchById',
  async (id, { rejectWithValue }) => {
    try {
      const { data } = await getProductById(id);
      return data.data;
    } catch (err) {
      return rejectWithValue(err.response?.data?.message || 'Product nahi
 mila!');
    }
  }
);

export const createProductThunk = createAsyncThunk(
```

```
  'products/create',
  async (productData, { rejectWithValue }) => {
    try {
      const { data } = await createProduct(productData);
      return data.data;
    } catch (err) {
      return rejectWithValue(err.response?.data?.message || 'Create failed!');
    }
  }
);

export const deleteProductThunk = createAsyncThunk(
  'products/delete',
  async (id, { rejectWithValue }) => {
    try {
      await deleteProduct(id);
      return id;  // Deleted product ki ID wapas bhejo
    } catch (err) {
      return rejectWithValue(err.response?.data?.message || 'Delete failed!');
    }
  }
);

// ■■ Slice ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const productSlice = createSlice({
  name: 'products',
  initialState: {
    items:         [],
    selectedProduct: null,
    loading:       false,
    error:         '',
    total:         0,
    totalPages:    1,
    currentPage:   1,
```

```
  },

  reducers: {
    clearSelectedProduct: (state) => { state.selectedProduct = null; },
    clearError:           (state) => { state.error = ''; },
  },

  extraReducers: (builder) => {
    builder
      // fetchProducts
      .addCase(fetchProducts.pending,   s => { s.loading=true;  s.error='
'; })
      .addCase(fetchProducts.fulfilled, (s, a) => {
        s.loading=false; s.items=a.payload.data;
        s.total=a.payload.total; s.totalPages=a.payload.totalPages;
        s.currentPage=a.payload.currentPage;
      })
      .addCase(fetchProducts.rejected,  (s,a) => { s.loading=false; s.err
or=a.payload; })

      // fetchProductById
      .addCase(fetchProductById.pending,   s => { s.loading=true; s.error
=''; s.selectedProduct=null; })
      .addCase(fetchProductById.fulfilled, (s,a) => { s.loading=false; s.
selectedProduct=a.payload; })
      .addCase(fetchProductById.rejected,  (s,a) => { s.loading=false; s.
error=a.payload; })

      // createProduct
      .addCase(createProductThunk.fulfilled, (s,a) => { s.items.unshift(a
.payload); })

      // deleteProduct
      .addCase(deleteProductThunk.fulfilled, (s,a) => {
        s.items = s.items.filter(p => p._id !== a.payload);
      });
  },
});

export const { clearSelectedProduct, clearError } = productSlice.actions;
```

```
// Selectors
export const selectProducts       = s => s.products.items;
export const selectSelectedProduct= s => s.products.selectedProduct;
export const selectProductsLoading= s => s.products.loading;
export const selectProductsError  = s => s.products.error;
export const selectTotalPages     = s => s.products.totalPages;

export default productSlice.reducer;
```

SECTION 9 — Components Redux Se Update

9.0 — HomeScreen — Redux Se Products

■ src/screens/HomeScreen.jsx

```
import { useEffect }           from 'react';
import { useDispatch, useSelector } from 'react-redux';
import { fetchProducts, selectProducts,
         selectProductsLoading, selectProductsError } from '../store/productSlice';
import ProductCard from '../components/ProductCard';

function HomeScreen() {
  const dispatch = useDispatch();

  // Redux store se state lo
  const products = useSelector(selectProducts);
  const loading  = useSelector(selectProductsLoading);
  const error    = useSelector(selectProductsError);

  useEffect(() => {
    // Thunk dispatch karo — API call hogi
    dispatch(fetchProducts());
  }, [dispatch]);

  if (loading) return <p style={{color:'#9B6DD4',textAlign:'center'}}>Loading...</p>;
  if (error)   return <p style={{color:'#FF6B6B',textAlign:'center'}}>{error}</p>;

  return (
    <div>
      <h1 style={{color:'#E6EDF3',marginBottom:'24px'}}>Latest Products</h1>
      <div style={{display:'grid',gridTemplateColumns:'repeat(auto-fill,minmax(220px,1fr))',gap:'20px'}}>
```

```
          {products.map(p => <ProductCard key={p._id} product={p} />)}
        </div>
      </div>
    );
  }
  export default HomeScreen;
```

9.1 — LoginScreen — dispatch setCredentials

■ src/screens/LoginScreen.jsx

```
  import { useState }              from 'react';
  import { useDispatch }            from 'react-redux';
  import { useNavigate }            from 'react-router-dom';
  import { setCredentials }         from '../store/authSlice';
  import { loginUser }              from '../services/api';

  function LoginScreen() {
    const dispatch  = useDispatch();
    const navigate  = useNavigate();
    const [email,    setEmail]    = useState('');
    const [password, setPassword] = useState('');
    const [loading,  setLoading]  = useState(false);
    const [error,    setError]    = useState('');

    const submitHandler = async (e) => {
      e.preventDefault();
      setLoading(true); setError('');
      try {
        const { data } = await loginUser(email, password);
        // Redux store update karo
        dispatch(setCredentials({ user: data.data, token: data.token }));
        navigate('/');
      } catch (err) {
        setError(err.response?.data?.message || 'Login failed!');
      } finally {
        setLoading(false);
```

```
    }
  };

  return (
    <form onSubmit={submitHandler} style={{maxWidth:'400px',margin:'40px auto'}}>
      <h2 style={{color:'#E6EDF3',marginBottom:'20px'}}>Login</h2>
      {error && <p style={{color:'#FF6B6B',marginBottom:'12px'}}>{error}</p>}
      <input type='email'    value={email}    onChange={e=>setEmail(e.target.value)}
        placeholder='Email'  style={{width:'100%',padding:'10px',marginBottom:'12px',
          background:'#1C2128',border:'1px solid #30363D',color:'white',borderRadius:'6px'}} />
      <input type='password' value={password} onChange={e=>setPassword(e.target.value)}
        placeholder='Password' style={{width:'100%',padding:'10px',marginBottom:'16px',
          background:'#1C2128',border:'1px solid #30363D',color:'white',borderRadius:'6px'}} />
      <button type='submit' disabled={loading}
        style={{width:'100%',padding:'12px',background:'#764ABC',
          color:'white',border:'none',borderRadius:'6px',cursor:'pointer',fontSize:'16px'}}>
        {loading ? 'Logging in...' : 'Login'}
      </button>
    </form>
  );
}
export default LoginScreen;
```

9.2 — Navbar — Redux Auth

■ src/components/Navbar.jsx — Redux Version

```
import { useDispatch, useSelector }   from 'react-redux';
import { Link, NavLink, useNavigate } from 'react-router-dom';
import { logout, selectUser,
```

```
         selectIsLoggedIn, selectIsAdmin } from '../store/authSlice';
import { selectTotalItems }             from '../store/cartSlice';

function Navbar() {
  const dispatch    = useDispatch();
  const navigate    = useNavigate();
  const user        = useSelector(selectUser);
  const isLoggedIn  = useSelector(selectIsLoggedIn);
  const isAdmin     = useSelector(selectIsAdmin);
  const totalItems  = useSelector(selectTotalItems);

  const logoutHandler = () => {
    dispatch(logout());   // Redux store clear karo
    navigate('/login');
  };

  return (
    <nav style={{display:'flex',justifyContent:'space-between',alignItems
:'center',
      padding:'12px 24px',background:'#161B22',borderBottom:'1px solid #3
0363D',
      position:'sticky',top:0,zIndex:100}}>
      <Link to='/' style={{color:'#9B6DD4',fontSize:'22px',fontWeight:'bo
ld',textDecoration:'none'}}>
        ■ ShopEase
      </Link>
      <div style={{display:'flex',gap:'16px',alignItems:'center'}}>
        <NavLink to='/cart' style={({isActive})=>({color:isActive?'#9B6DD
4':'#8B949E',textDecoration:'none'})}>
          ■ {totalItems > 0 && <span style={{background:'#FF6B6B',color:'
white',
            borderRadius:'50%',fontSize:'11px',padding:'2px 6px',marginLe
ft:'4px'}}>{totalItems}</span>}
        </NavLink>
        {isAdmin && <NavLink to='/admin/products' style={({isActive})=>({
color:isActive?'#9B6DD4':'#8B949E',textDecoration:'none'})}>Admin</NavLin
k>}
        {isLoggedIn ? (
```

```
          <><span style={{color:'#E6EDF3'}}>■ {user?.name?.split(' ')[0]}
</span>
            <button onClick={logoutHandler} style={{background:'none',bor
der:'1px solid #FF6B6B',
              color:'#FF6B6B',padding:'4px 12px',borderRadius:'6px',curso
r:'pointer'}}>Logout</button></>
        ) : (<NavLink to='/login' style={{color:'#8B949E',textDecoration:
'none'}}>Login</NavLink>)}
      </div>
    </nav>
  );
}
export default Navbar;
```

SECTION 10 — Summary, Practice & Next Steps

10.0 — Chapter 8 Ka Recap

Redux	Predictable state container — ek global store
Redux Toolkit (RTK)	Official Redux simplifier — verbose code khatam
configureStore	Store banana — reducers combine karna
Provider	index.js mein wrap — saari app ko store milta hai
createSlice	name + initialState + reducers = actions + reducer
action.payload	dispatch(action(data)) mein bheja hua data
Immer	RTK mein built-in — direct mutation allowed!
useSelector	Store se state read karo — selector function
useDispatch	dispatch() function milta hai — actions bhejo
cartSlice	addItem, removeItem, updateQuantity, clearCart + localStorage
authSlice	setCredentials, logout + localStorage sync
createAsyncThunk	Async API calls — pending/fulfilled/rejected auto dispatch
extraReducers	Thunk states handle karna — builder pattern
productSlice	fetchProducts, fetchProductById, create, delete thunks
Selectors	selectProducts, selectUser etc. — named exports for reuse
Redux DevTools	Chrome extension — action log, state diff, time travel

10.1 — Practice Exercises

Easy Level

1. @reduxjs/toolkit aur react-redux install karo
2. store.js banao — cartReducer, authReducer add karo
3. index.js mein Provider wrap karo

4. Redux DevTools install karo — store check karo

■ Medium Level

1. cartSlice banao — addItem, removeItem, clearCart
2. authSlice banao — setCredentials, logout
3. LoginScreen mein dispatch(setCredentials) karo
4. Navbar mein useSelector se user aur totalItems lo

■ Challenge Level

1. productSlice banao — fetchProducts thunk ke saath
2. HomeScreen mein dispatch(fetchProducts()) karo
3. ProductScreen mein dispatch(fetchProductById(id)) karo
4. CartScreen — useSelector se items lo, dispatch karo
5. Context API files hata do — poora Redux pe migrate karo
6. Redux DevTools mein actions trace karo

10.2 — Chapter 9 Ka Preview

Frontend Pages — Product List, Detail, Search & Filter

- ■ HomeScreen — responsive product grid with loading skeleton
- ■ ProductScreen — full detail page with image, rating, reviews
- ■ SearchBox — keyword search functionality
- ■ Filter — category, price range filter
- ■ Paginate — page by page navigation
- ■ Rating component — star display
- ■ Review system — add aur display reviews
- ■ Responsive design — mobile friendly

Redux Master Ho Gaye! ■■

Aaj tumne Redux Toolkit ka poora system seekha — Store, Slices, Actions, Reducers, useSelector, useDispatch, aur createAsyncThunk se API calls. Yeh wahi tools hain jo badi companies apne production apps mein use karti hain! **Chapter 9 mein poore frontend pages complete karenge — ShopEase nazar aayega!**